REINFORCE and Q-learning

Learning useful tensor-network clusters 23 October 2026

Both methods learn the same construction decisions. Starting with current domains, the learner chooses a target/open variable, adds cores, chooses further outputs and finishes. The completed cluster is contracted under those domains, then GAC measures its effect. An individual core addition need not prune anything.

The difference is the **meaning and training target of the scores**, not just whether they are normalized. REINFORCE learns action preferences. Q-learning fits predictions of eventual reward.

Variables used below

Symbol	Meaning
s_t, a_t	Construction state at step t (network, domains, selected cores/outputs and phase), and the action taken.
T, $\mathcal{A}(s)$	Number of actions in a completed construction; legal actions at state s.
θ, η	Trainable network parameters; learning rate.
$z_\theta(s,a)$	Actor's unnormalized preference score, called a logit. Its absolute value has no reward meaning.
$\pi_\theta(a \mid s)$	Probability of choosing action a in state s.
$Q_\theta(s,a)$	Predicted reward obtainable by taking a and completing the cluster well. It is not a probability.
R	Terminal reward measured from the cluster. Our Q learner uses normalized log-domain reduction; contradiction gives one, no-op/skip/rejection zero. Exact calculation: page 5.
b; $c > 0$	REINFORCE reward baseline and positive scale. Computed statistics, held constant during backpropagation; estimation rules: page 4.
θ^-, y_t	Frozen target-network parameters; numerical target for a Q prediction.

1. REINFORCE: change probabilities using the observed reward

For the basic actor, probabilities are normalized scores:

$$\pi_\theta(a \mid s) = \frac{\exp z_\theta(s,a)}{\sum_{u\in\mathcal{A}(s)} \exp z_\theta(s,u)}.$$

The main loss for one sampled construction is

$$\boxed{L_{\text{policy}} = -\frac{R-b}{c}\sum_{t=0}^{T-1} \log \pi_\theta(a_t \mid s_t).}$$

If $R > b$, the update favors the sampled actions; if $R < b$, it disfavors them. The actor's scores are *not* trained to equal R. Both a useful and a useless construction can contain actions that must be judged relative to their available alternatives.

2. Q-learning: fit predictions to reward-based targets

For one recorded state/action, the basic undiscounted loss and target are

$$\boxed{L_Q = \text{Huber}(Q_\theta(s_t,a_t) - y_t)\,,} \qquad y_t = \begin{cases} R, & t = T-1, \\ \max\limits_{u\in\mathcal{A}(s_{t+1})} Q_{\theta^-}(s_{t+1},u), & t < T-1. \end{cases}$$

The target uses the **next state's best predicted action**, not the current prediction. No gradient flows through y_t. Batch losses average these terms. Our extra target rules are explained on page 3. Both methods use backpropagation, e.g. $\theta \leftarrow \theta - \eta\nabla_\theta L$ with plain gradient descent.

A numerical example

One observation, two different reference points

Suppose a partial cluster is ready to finish, but another output variable could still be retained. There are two legal actions: $F =$ **finish now**, and $O =$ **open another variable and continue**. We take F, contract the cluster and run GAC. Its measured reward is $R = 0.2$. This observation gives no direct measured reward for O.

Take two hypothetical models with identical initial outputs $(0.8, 0.8)$, whose meanings differ:

$$\begin{aligned} &\text{Actor logits:} && z_F = z_O = 0.8 && \Rightarrow\ \pi(F) = \pi(O) = 0.5, \\ &\text{Q predictions:} && Q_F = Q_O = 0.8 && \Rightarrow\ \text{both predict reward } 0.8. \end{aligned}$$

For arithmetic, set $b = 0$, $c = 1$ and $\eta = 0.1$. These are illustrative choices, not the current learner's defaults (page 4). Omit entropy/imitation, treat outputs as independent parameters and use plain gradient descent. Real training shares parameters and uses Adam. Show only this decision's loss contribution.

REINFORCE increases the probability of finishing

Because the observed reward exceeds the baseline, the loss is

$$L_{\text{policy}} = -(0.2 - 0)\log(0.5) = 0.138629.$$

For a softmax, the gradients of this loss with respect to the two logits are

$$\frac{\partial L}{\partial z_F} = 0.2(0.5 - 1) = -0.1, \qquad \frac{\partial L}{\partial z_O} = 0.2(0.5) = +0.1.$$

A gradient-descent step gives

$$(z_F, z_O) \leftarrow (0.8, 0.8) - 0.1(-0.1, +0.1) = (0.81, 0.79),$$

$$\boxed{\pi(F) = \frac{e^{0.81}}{e^{0.81} + e^{0.79}} \approx 0.505.}$$

The probability of finishing rises from 50% to about 50.5%. If the baseline were above 0.2, the update would instead decrease it.

Q-learning lowers its overoptimistic prediction for finishing

Because F terminates construction, its target is the measured reward: $y = 0.2$. The error is $Q_F - y = 0.8 - 0.2 = 0.6$. For errors of magnitude at most one, our Huber loss is half the squared error:

$$L_Q = \tfrac{1}{2}(0.6)^2 = 0.18, \qquad \frac{\partial L_Q}{\partial Q_F} = 0.6.$$

Therefore

$$\boxed{Q_F \leftarrow 0.8 - 0.1(0.6) = 0.74, \qquad Q_O = 0.8.}$$

The prediction moves down toward 0.2. $Q_O = 0.8$ remains an unverified prediction, not evidence that opening another variable is better. With shared parameters, this update could indirectly change Q_O too.

What this example shows

	Compared against	Consequence of observing 0.2
REINFORCE	Baseline $b = 0$	Finishing becomes more likely.
Q-learning	Prediction $Q_F = 0.8$	Predicted reward decreases.

Both use gradients. Update magnitudes are **not a performance comparison**. Applying a softmax to Q-values cannot make the two training targets equivalent.

Earlier choices and our implementation

How delayed pruning credit reaches core additions

A successful completion gives earlier actions an achievable return

Suppose an earlier action added core A and the recorded completion earned $R = 0.2$. Intermediate rewards were zero. With no discount, this path demonstrates an achievable return of 0.2 from the earlier action.

REINFORCE uses the same construction reward in the sampled decisions' log-probability terms. It can therefore reward an early core choice even though that choice alone caused no pruning.

Our Q learner uses 0.2 as an achievable lower bound for an intermediate target. Suppose its current prediction for adding A is 0.1, and the next state's two legal actions have target-network values 0.6 and 0.1. Ignoring the optional farther-step target in this example:

$$y = \max\{\underbrace{0.6}_{\text{best next prediction}}, \underbrace{0.2}_{\text{measured completion}}\} = 0.6, \qquad L_Q = \tfrac{1}{2}(0.1 - 0.6)^2 = 0.125.$$

The earlier action is trained toward 0.6, not forced to equal the recorded completion's 0.2. Another continuation might be better. However, **0.6 is only a prediction**; the measured 0.2 is the verified achievable return. Prediction errors can make the maximum overoptimistic.

Exact extra target rules in the current Q code

Define the bounded best next-action prediction

$$V^-(s) = \mathrm{clip}_{[0,1]}\left(\max_{u \in \mathcal{A}(s)} Q_{\theta^-}(s, u)\right).$$

Here clipping limits a number to $[0, 1]$, the known Q reward range. For a recorded trace, our targets are

$$y_t = \begin{cases} R, & t = T - 1, \\ V^-(s_{t+1}), & t < T - 1 \text{ and } R = 0, \\ \max\{V^-(s_{t+1}),\ R,\ V^-(s_{t+n})\}, & t < T - 1 \text{ and } R > 0. \end{cases}$$

The last term is included only when $n > 1$ and $t + n < T$. The backup length n is configurable (default eight). These maxima do *not* include the current prediction $Q_\theta(s_t, a_t)$. Failed finishes get exact zero targets; earlier choices can retain valuable alternative completions.

The prediction error is $e = Q_\theta - y$. The full loss is $\mathrm{Huber}(e) = e^2/2$ for $|e| \le 1$, and $|e| - 1/2$ otherwise. Q updates sample successes and failures from replay. The *target network* is a frozen copy supplying predictions, not a teacher example.

What "teacher" and "imitation" mean

A teacher is a **measured useful cluster**, with its cores and outputs, used as an example. Structural proposals, their modifications and failed-value propagation explanations can supply examples; contraction and GAC verify usefulness. A teacher need not be a person or another network.

The earlier teacher-assisted policy adds an imitation loss. If $\mathcal{A}^+(s)$ contains actions continuing toward verified useful examples, a basic imitation term is

$$L_{\text{teacher}}(s) = -\log \sum_{a \in \mathcal{A}^+(s)} \pi_\theta(a \mid s).$$

Our code weights starting and continuation decisions and can teach best-known completions at the student's own prefixes. Actor losses also include entropy regularization for exploration. Page 2 omits these terms and holds R fixed; historical experiments used different reward recipes.

Code basis: `qlearn.py` (`q_loss`); `learn.py` (policy/teacher losses); `train.py` (original policy); `model.py` (probabilities/traces). All are in `experiments/cluster_selection/reinforcement/`. Worked numbers are illustrative, not experimental solver results.

How the baseline and scale are chosen

Computed from rewards, with fixed design choices

b and c are computed statistics, not independently tuned numbers for each construction. Their estimation rules contain fixed design choices. Neither is a learned neural-network output in these implementations. Q-learning uses neither b nor c in its loss.

Baseline b: compare with other constructions from the same state

In the teacher-assisted REINFORCE learner, `learn.py`, let K be the number of sampled constructions with valid rewards from the same current domain state. Their rewards are $R_1, \ldots, R_K$. Each construction i gets its own baseline:

$$\boxed{b_i = \begin{cases} \dfrac{1}{K-1}\displaystyle\sum_{j \neq i} R_j, & K > 1, \\ 0, & K = 1. \end{cases}}$$

The comparison excludes construction i itself. Its normalized advantage is $(R_i - b_i)/c$: positive means better than the other sampled constructions, negative means worse. Invalid rewards, such as timed-out DFS comparisons, are excluded; with no valid rewards, the reward-based actor term is omitted.

Scale c: normalize the size of the policy update

The code maintains a running mean-square reward statistic v, initialized to 0.01. Before an update, it uses

$$\boxed{c = \max(0.05, \sqrt{v}).}$$

After processing the valid rewards, the statistic is updated as

$$\boxed{v \leftarrow 0.95v + 0.05\frac{1}{K}\sum_{i=1}^{K} R_i^2.}$$

Thus the initial scale is $c = 0.1$. It is a smoothed **root-mean-square reward scale**, not a standard deviation: rewards are not centered before squaring. The floor prevents division by a very small number. Since $c > 0$, it changes update size, not whether an action is encouraged or discouraged.

Example using the actual estimation rule

Suppose three constructions from one state earn $(0.2, 0.05, 0)$, and the running statistic is still $v = 0.01$. For the first construction:

$$b_1 = \frac{0.05 + 0}{2} = 0.025, \qquad c = 0.1, \qquad \frac{R_1 - b_1}{c} = \frac{0.2 - 0.025}{0.1} = 1.75.$$

The policy update favors its sampled actions. A positive reward need not give a positive advantage: if the other constructions performed better, its baseline would exceed its reward.

Which parts are hyperparameters?

The choice of estimator, smoothing factor 0.95, initialization 0.01 and floor 0.05 are **design hyperparameters**. These values are currently fixed constants in the code, rather than separate command-line options. The resulting b_i and c adapt to observed rewards. The learning rate remains another, separate hyperparameter.

Historical distinction. The oldest `train.py` uses a rolling-average baseline starting at zero, smoothing factor 0.9, scale floor one, and a moment statistic starting at one based on squared deviations from that baseline. The formulas above describe `learn.py`; page 2 is intentionally simplified.

Reward choice is separate from the learning rule. Earlier mixed-reward actors combined pruning, bonuses and computational costs, optionally adding a search-savings term. The DFS-only actor instead uses $R = N_{\text{before}} - N_{\text{after}}$: the difference in completed ordinary-MAC DFS visits, which can be negative. Timed-out comparisons are excluded. The next page gives the current Q learner's pruning reward.

How the reward is calculated

Current Q-learning objective: measured pruning

The reward is measured, not predicted. First close the current domains under ordinary GAC. Let D_v be variable v's domain at this point. Construct and contract a cluster conditioned on these domains, add its summary temporarily, and run GAC. Let D'_v be the resulting domain. The reward includes all propagation caused by this operation, across all variables, not just the cluster's open variables.

1. Measure reduction in log-domain sizes

For a noncontradictory result, define

$$\boxed{H(D) = \sum_v w_v \log |D_v|, \qquad \Delta H = \sum_v w_v \log \frac{|D_v|}{|D'_v|}.}$$

$|D_v|$ means the number of currently allowed values. $H(D)$ is a weighted measure of remaining domain uncertainty; ΔH is the reduction. Singleton domains contribute zero. These are log-cardinality measures, **not the entropy of the full correlated solution distribution**. The code uses natural logarithms; a different consistent log base gives the same ratio.

2. Weight primary and auxiliary variables

The data designate primary reward variables through `reward_colors`; all other variables are auxiliary. Let n_p and n_a be their respective counts in the network. The code uses

$$w_v = \begin{cases} 1, & v \text{ is a primary reward variable,} \\ \min\left(1, \dfrac{n_p}{\max(1, n_a)}\right), & v \text{ is auxiliary.} \end{cases}$$

This keeps a large auxiliary encoding from overwhelming the primary variables' contribution. It is a generic role-based weighting, not a Sudoku inference rule. If no subset is designated, all variables count as primary.

3. Normalize the measured reduction

$$\boxed{R = \begin{cases} 1, & \text{contraction and GAC detect a contradiction,} \\ \mathrm{clip}_{[0,1]}\left(\dfrac{\Delta H}{H(D)}\right), & \text{accepted, noncontradictory result and } H(D) > 0, \\ 0, & \text{otherwise.} \end{cases}}$$

A no-op, skip or rejected/over-budget construction earns zero. $H(D) = 0$ also gives zero. There is no separate solve bonus, time/work penalty, treewidth cost or DFS-node term in this Q reward. Hard contraction work and tensor-size limits still constrain legal/affordable constructions. A contradiction in a known satisfiable training state raises an error.

A concrete source for the example reward $R = 0.2$

Suppose all four variables have weight one, and contraction plus GAC changes their domain sizes as follows:

$$(4, 2, 2, 2) \longrightarrow (2, 2, 2, 2).$$

Then

$$H(D) = \log 4 + 3 \log 2 = 5 \log 2, \qquad \Delta H = \log(4/2) = \log 2, \qquad \boxed{R = \frac{\log 2}{5 \log 2} = 0.2.}$$

This is 20% of this weighted log-domain measure removed, not a probability of success or a promise of 20% fewer DFS nodes. It supplies the illustrative reward used on page 2.

Code basis: `qlearn.py` calls `learn.py:measure` with `objective="pruning"`. This objective uses $H(D)$ itself as the denominator, rather than the $\max(1, H(D))$ normalization in the older mixed reward.